SHoEbox Runtime Optimization Update

6 November 2026 | Compared with bd0842e

The main measured gain is lower expert-gradient allocation; throughput changes are mixed. Comparison is against the immediately preceding optimized release, bd0842e, with identical initialized weights and inputs. Model capacity and feedback rounds are preserved.

Model	Workload	Before (s)	After (s)	Speedup
2.56M	Forward + backward	3.570	3.746	0.95x
2.56M	Inference segment	1.034	1.073	0.96x
2.56M	Generation	1.849	1.972	0.94x
24.91M	Forward + backward	4.156	4.277	0.97x
24.91M	Inference segment	1.293	1.345	0.96x
24.91M	Generation	2.497	2.355	1.06x

For the 24.91M configuration, one byte position across eight streams now creates 48 whole-bank gradient buffers instead of 168. Cumulative bank allocations fall from 141.75 MiB to 40.50 MiB (71.4% less). This measures allocated bytes, not peak process RAM.

Implemented changes

Expert gradients. Large banks combine selected-expert backward operations, accumulate compact row derivatives and allocate one dense gradient per bank per byte event. A native row accumulator avoids sparse-to-dense overhead. Small banks keep the cheaper existing path. AdamW still receives dense gradients.

Recurrent backward. Large-bank models retain MinKron forward arithmetic and reuse saved decay, norm, score and update terms in a fused CPU backward. A differentiable PyTorch path handles higher derivatives. Small models retain the existing cheaper path; systems without the extension have a scripted fallback.

Routing projections. Each forward computes the prototype/message matrix product once and reuses it across bytes and rounds. The trainable projection remains in the gradient graph.

Caching, batching and verification

Generation caching. Both generation interfaces retain normalized routing tables, symbol keys, lace templates and decay constants across bytes. Parameter and buffer versions, replacement, dtype/device changes, checkpoint loads and autocast mode invalidate the cache.

Independent stages. Large-bank training batches compatible full-activity contextualizers and pilot projections. Nodes keep separate weights and owned state storage. Inference, partial masks and registered hooks use the individual path; stage snapshots and commit order are preserved. The batching flag can also be enabled explicitly on smaller models.

Repeated calculations. Gradient norms and task dot products are reduced once per parameter and aggregated for both reporting views. The observational lace pass reuses probabilities already produced by routing.

Validation and measurement

Validation: 433 unique regression checks pass after the two batching fixes. Checks include FP64 outputs and derivatives, finite differences and higher derivatives, repeated expert selections, AdamW moments, independent backwards, cache invalidation, mixed masks, resets, hooks, state storage, checkpoint continuation and generation resume.

Projection reassociation and batched operations change FP32 rounding. The regression checks compare that drift against an FP64 reference while checking discrete routes in the tested cases. The exact FP64 derivative checks remain strict.

Protocol: PyTorch 2.15.0+cpu, AVX512, one CPU thread, 8 streams x 16 byte positions, two feedback rounds, 5 interleaved measured pairs after warmup. Generation uses one stream and 32 new bytes. The 24.91M graph uses the existing eleven-node configuration and the byte-stream policy, which leaves four memory producers inactive. These are synthetic throughput checks, not new language-training results.

The native CPU extension builds locally on Linux when a C++20 compiler is available; it requires no downloads or Ninja. Initial compilation is excluded from timings. SHOEBOX_NATIVE=0 selects the portable fallback. model.batch_independent=False disables stage batching; model.set_execution_backend("reference") selects the preserved reference implementation.

Reproduction commands and full measurements are supplied in the Markdown report and source archive.